

REPRESENTAÇÃO E MANIPULAÇÃO DE CONHECIMENTO

Da IA Clássica aos Agentes Modernos de IA

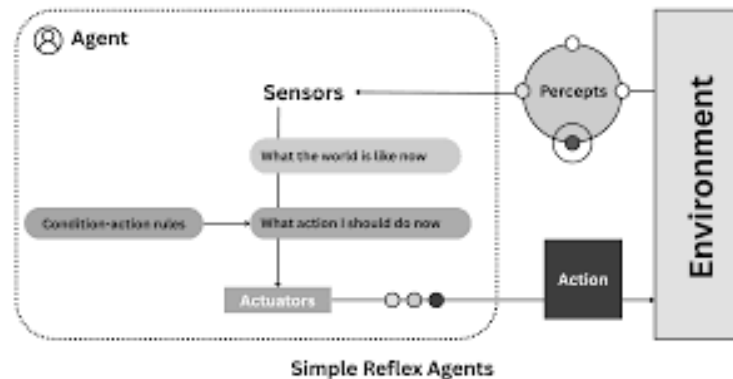
Rogério de Oliveira

2026-05-12

O DESAFIO DE MARTE

- **Cenário:** O Rover precisa coletar uma amostra em uma cratera, mas sua bateria está em 30% e uma tempestade de areia se aproxima.
- **A Lógica (Diagnóstico):** "Se o sensor de opacidade $> 80\%$, então tempestade detectada."
- **O Planejamento (Estratégia):** "Qual a sequência de movimentos para chegar à cratera e voltar à base antes da bateria acabar?"
- **O Escalonamento (Recursos):** "Tenho 2 braços robóticos. Posso perfurar o solo e transmitir dados simultaneamente ou a energia não suportará?"

AGENTES DE IA



- Um agente de IA é um sistema de software inteligente capaz de perceber seu ambiente, raciocinar e executar tarefas complexas de forma autônoma para atingir um objetivo específico.
- Agentes: Representação do Conhecimento → Ação

AGENDA

- **Lógica Proposicional**
- **Lógica de Predicados:** Generalizando para Objetos e Relações.
 - Sistemas Especialistas
- **Planejamento Automatizado:** Algoritmo para criar estratégias
 - Busca, STRIPS
- **Escalonamento (Scheduling):** Restrições de tempo e recursos
- **Agentes de IA Modernos**

LÓGICA PROPOSICIONAL

- **Proposições (Átomos):** Sentenças binárias (V ou F): Oleo_Baixo
- **Conectores:** $\neg$ (Não), $\wedge$ (E), $\vee$ (Ou), $\rightarrow$ (Implicação).
- **Exemplo:** $(\text{Motor_Ruido} \wedge \text{Oleo_Baixo}) \rightarrow \text{Alerta_Aceso}$.
- **Inferência:** Modus Ponens ("modo de afirmar")

Se $p \rightarrow q$ e p é verdade, logo q é verdade

TABELAS VERDADE E INFERÊNCIA

$$p \rightarrow q$$

p	q	p→q	

0	0	1	
1	0	0	
0	1	1	
1	1	1	

Modus Ponens (definição formal)

Tautologia

Contradição

Equivalência de implicação

Modus Tollens

Leis de Morgan

Falácia afirmar o conseqüente

$$[(p \rightarrow q) \wedge p] \rightarrow q$$

$$p \vee \neg p$$

$$p \wedge \neg p$$

$$\neg(p \wedge \neg q)$$

$$\neg q \rightarrow \neg p$$

$$\neg p \vee \neg \neg q$$

$$q \rightarrow p$$

IA: Se a base de conhecimento tem $p \rightarrow q$ e o sensor diz que p é verdade, inferimos que q é verdade automaticamente

Inferência consiste em derivar novos fatos a partir de conhecimento previamente representado.

LIMITAÇÕES DA LÓGICA PROPOSICIONAL

Pouca Expressividade

- Representa Objetos individuais.
- Não representa Relações !
- Não permite Quantificação.

Como dizer 'Todo Aluno é Inteligente' sem criar 50 variáveis?

LÓGICA DE PREDICADOS (1ª ORDEM)

Estrutura Interna

- **Objetos:** Constantes (João, BlocoA) e Variáveis (x , y).
- **Predicados:** Propriedades ($Mortal(x)$) e Relações ($Sobre(A, B)$).

Quantificadores

- $\forall$ (Universal): 'Para todos...' $\forall x (Humano(x) \rightarrow Mortal(x))$
- $\exists$ (Existencial): 'Existe ao menos um...' $\exists x (Sensor(x) \wedge Ativo(x))$

APLICAÇÕES DE PREDICADOS

Exemplo "Armazém" Inteligente:

- **Regra:** $\forall x \forall y (\text{Pesado}(x) \wedge \text{Em_Cima}(x, y) \rightarrow \text{Fragil}(y))$
- **Fato:** $\text{Pesado}(\text{Motor}) \wedge \text{Em_Cima}(\text{Motor}, \text{Caixa01})$
- **Dedução:** $\text{Fragil}(\text{Caixa01})$

SISTEMAS ESPECIALISTAS (SE)

- Buscam emular especialistas humanos.
- Arquitetura
 - Base de Conhecimento (Regras SE-ENTÃO)
 - Motor de Inferência (Aplica a lógica).
- Encadeamento para Frente vs. Para Trás.
- **Exemplos:** CLIPS, MYCIN, PROSPECTOR

PYKNOWN: SÓCRATES É MORTAL

```
class Humans(KnowledgeEngine):
    @Rule(Fact('human'))
    def mortal(self):
        self.declare(Fact('mortal'))

    @Rule(Fact('socrates'))
    def human(self):
        self.declare(Fact('human'))

    def factz(self, l):
        for x in l:
            self.declare(x)
```

```
ex = Humans()
ex.reset()
ex.factz([Fact('socrates')])
ex.run()
ex.facts
```

```
FactList([(0, InitialFact()),
          (1, Fact('socrates')),
          (2, Fact('human')),
          (3, Fact('mortal'))])
```

PyKnow exemplo

IDEIA DE PROJETO 1

- Buscam emular especialistas humanos.
- Arquitetura
 - Base de Conhecimento (Regras SE-ENTÃO) ← Especialista Humano
← LLM, PyKnow Rules
- Motor de Inferência (Aplica a lógica).
- Encadeamento para Frente vs. Para Trás.

O QUE É PLANEJAMENTO EM IA?

- Planejar é basicamente escolher "o que fazer" antes de "fazer".
- Um algoritmo de busca simples (pense no Waze) encontra um caminho entre dois pontos, o **Planejamento Automatizado** é uma busca que encontra uma sequência de ações lógicas para transformar o estado do mundo.

O desafio do planejamento é que o número de combinações de ações possíveis cresce exponencialmente.

PLANEJAMENTO AUTOMATIZADO

- **O Problema de Planejamento:** Dado um estado inicial S_0 , um conjunto de ações A e um objetivo G , encontrar uma sequência de ações que leve a G

$$S_0 \rightarrow G.$$

- **Em uma busca comum:** O estado é uma "caixa preta" (ex: X, Y).
- **No planejamento:** O estado é "transparente" (composto por fatos lógicos que a IA entende e manipula).
- **Ciclo do Agente IA:** Perceber $\rightarrow$ Planejar $\rightarrow$ Atuar $\rightarrow$ Corrigir.

PLANEJAMENTO: O MODELO STRIPS

De define o mundo através de:

Estado: Um conjunto de literais (fatos verdadeiros no momento).

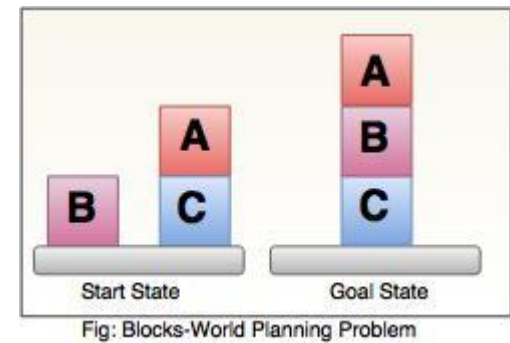
Objetivo: Onde queremos chegar.

Ações (Operadores): O coração do sistema. Cada ação possui:

- **Pré-condições:** O que deve ser verdade para a ação ocorrer.
- **Efeito (Add List):** O que passa a ser verdade após a ação.
- **Efeito (Delete List):** O que deixa de ser verdade.

Assume que **tudo o que não é explicitamente alterado por uma ação permanece igual.**

STRIPS: EXEMPLO



Imagine que queremos colocar o Bloco A sobre o Bloco B.

- **Ação:** $Empilhar(A, B)$
- **Pré-condições:** $Sobre(A, Mesa) \wedge Limpo(A) \wedge Limpo(B)$
- **Efeitos:**
 - **Adicionar (Add):** $Sobre(A, B) \wedge Limpo(Mesa)$
 - **Remover (Delete):** $Sobre(A, Mesa) \wedge Limpo(B)$

Exemplo unified-planning

EMPILHAR(A,B), O QUE A IA FAZ?

Ela olha para o Objetivo, olha para o Estado Inicial, e usa um algoritmo de busca (como uma Busca em Grafo, A*, etc.) para testar as ações STRIPS. A Busca: *"Se eu aplicar a Ação 1, chego no objetivo? Não. E se eu aplicar a Ação 2?"*.

Quando ela encontra uma sequência de ações que transforma o Estado Inicial no Objetivo, ele gera um **Plano**.

Apenas o plano final é enviado para os motores físicos do robô executarem.

PLANEJAMENTO COMO BUSCA EM ESPAÇO DE ESTADOS

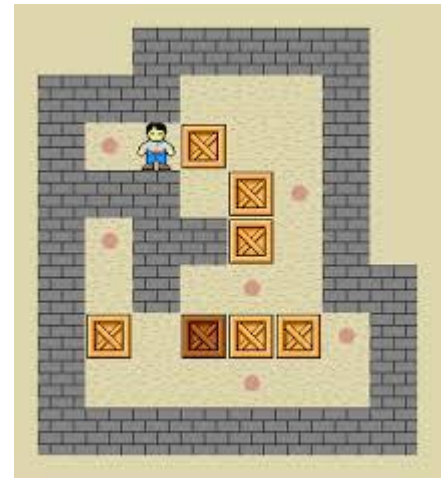
- **Analogia:** A^* é o Motor, STRIPS é o Combustível.
- **Nós:** Estados lógicos (ex: Mundo dos Blocos).
- **Arestas:** Ações válidas validadas pelo STRIPS.
- **Heurística (h):** Estimativa de distância até o objetivo.

Expansão de Nós: Verificar quais ações STRIPS têm pré-condições satisfeitas. Aplicar a "Add List" e "Delete List" para gerar os nodos filhos.

SOKOBAN

Configuração Inicial (S0):

```
[ R ] [ . ] [ . ]  
[ . ] [ B ] [ . ]  
[ . ] [ . ] [ T ]
```



1. Estado 0 (S0): R em (1,1), B em (2,2)

- Ação: Mover Direita → S1: R(1,2), B(2,2)
- Ação: Mover Baixo → S2: R(2,1), B(2,2)

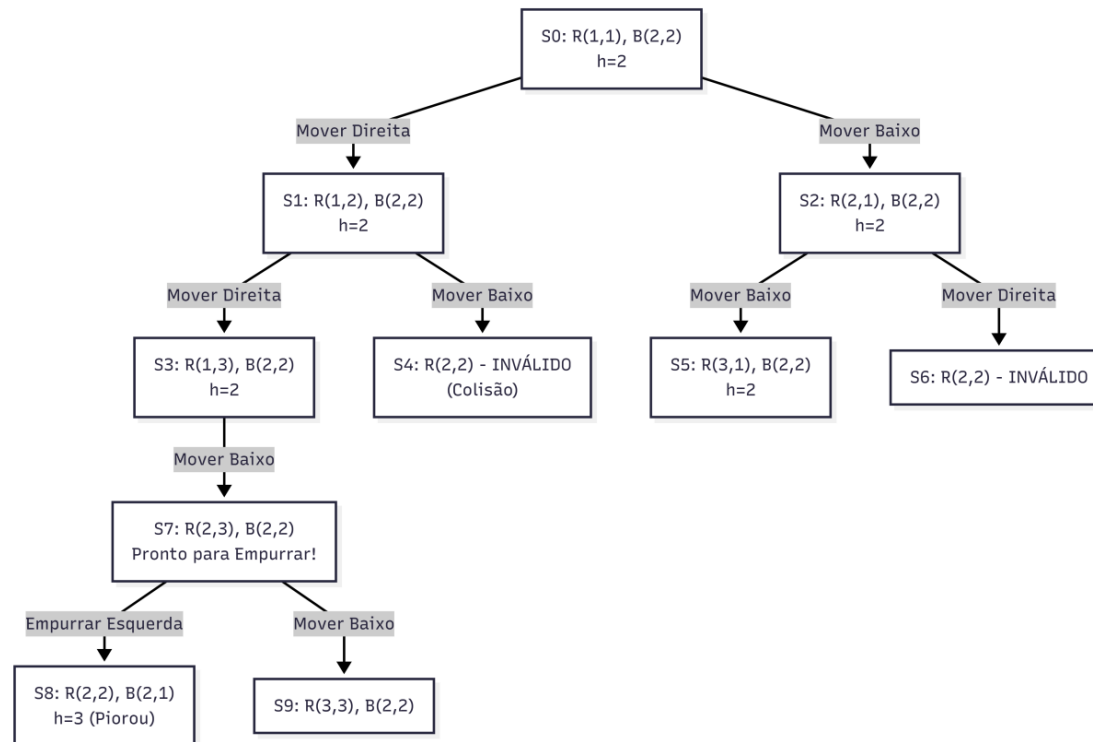
2. Expansão de S1:

- Ação: Mover Baixo → S3: R(2,2) -- **BLOQUEADO!** (Pré-condição falhou: B está em 2,2 e não há espaço para empurrar para baixo, pois 3,2 está livre mas o robô estaria em cima de B).
- Ação: Mover Direita → S4: R(1,3), B(2,2)

3. Caminho para o Objetivo:

- O A* favorece caminhos onde a distância de Manhattan entre B e T diminui.

SOKOBAN



ESCALONAMENTO

- **Planejamento (STRIPS)**

Foco: Causalidade e Pré-condições.

"Quais ações me levam ao objetivo?"

- **Escalonamento (Scheduling):**

Foco: Recursos limitados e Janelas de tempo.

"Quem faz o quê, em qual máquina e a que horas?"

- **Analogia:** Planejar é saber que o bolo precisa ser assado após ser batido. Escalonar é saber que o forno só suporta 2 bolos por vez e leva 40 minutos.

EXEMPLO DE PROBLEMA

Exemplo de problema

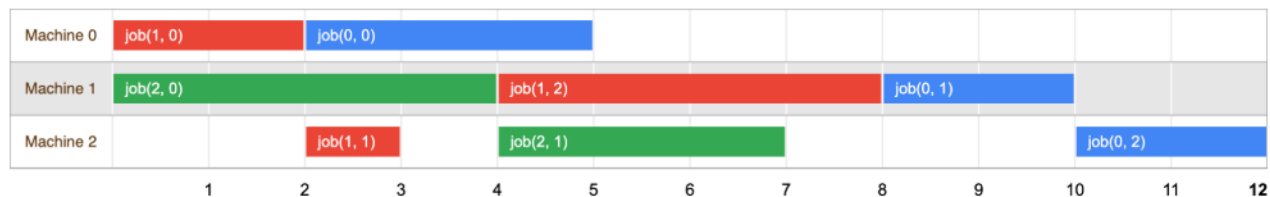
Abaixo está um exemplo simples de um problema de loja de empregos, em que cada tarefa é rotulada por um par de números (m, p) , em que m é o número da máquina que a tarefa precisa ser processado em e p é o *tempo de processamento* da tarefa — o tempo necessário. A numeração de jobs e máquinas começa em 0.

- vaga 0 = $[(0, 3), (1, 2), (2, 2)]$
- vaga 1 = $[(0, 2), (2, 1), (1, 4)]$
- vaga 2 = $[(1, 4), (2, 3)]$

No exemplo, o job 0 tem três tarefas. A primeira, $(0, 3)$, precisa ser processada na máquina 0 em 3 unidades de tempo. A segunda, $(1, 2)$, precisa ser processada máquina 1 em 2 unidades de tempo e assim por diante. Ao todo, há oito tarefas.

Uma solução para o problema

Uma solução para o problema do posto de trabalho é uma atribuição de um horário de início para cada que atenda às restrições acima. O diagrama abaixo mostra uma possível solução para o problema:



CSP: TABULEIRO DE RESTRIÇÕES

- **Variáveis (V):** Atividades ou tarefas (T_1 , T_2 , ...).
- **Domínios (D):** Espaços de tempo ou recursos disponíveis.
- **Restrições (C):**
 - *Unárias:* "A tarefa T_1 deve acabar antes das 10h".
 - *Binárias:* "A tarefa T_1 e T_2 não podem usar a mesma máquina".
- *Backtracking Search* com poda (parar assim que uma regra for violada).

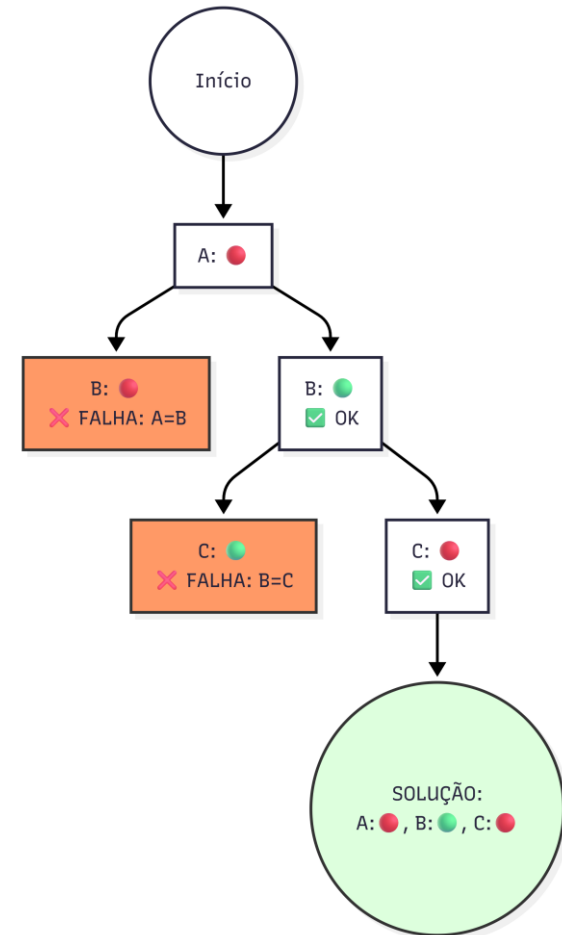
BACKTRACKING NO PROBLEMA DAS CORES (A-B-C)

Imagine o mapa linear: **A — B — C**.

Regra: Cores diferentes para vizinhos.

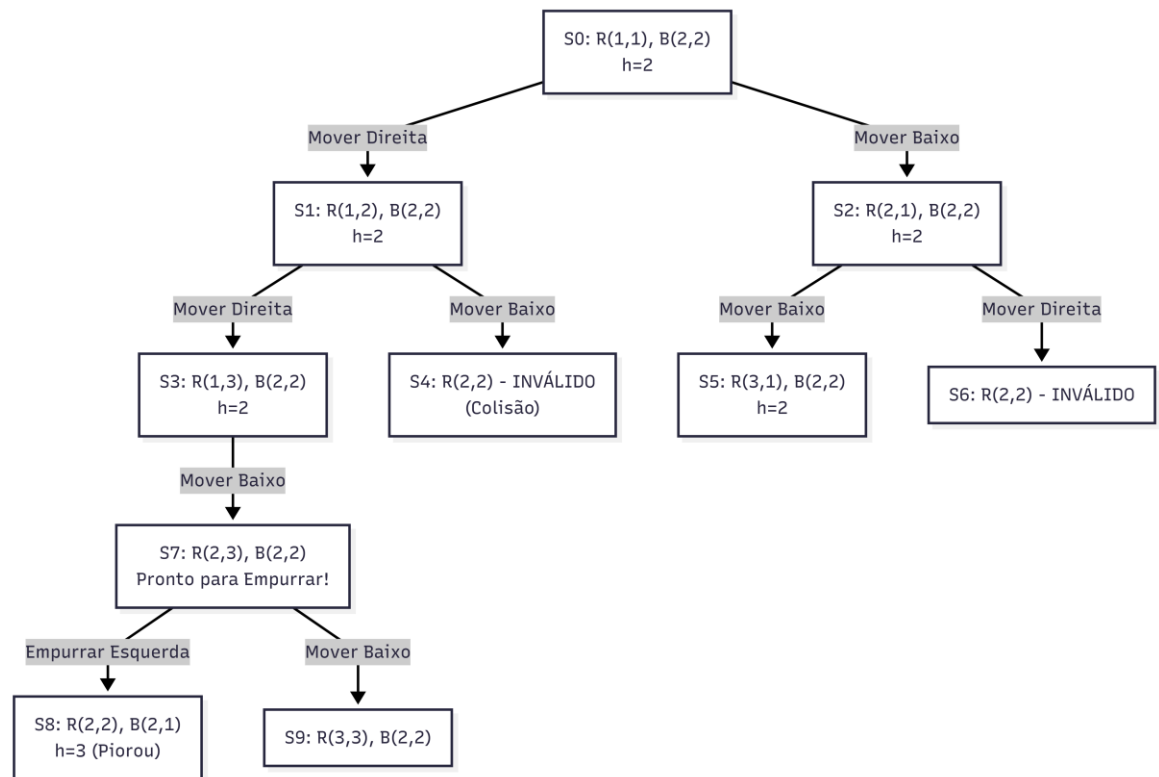
Domínio: {  Vermelho,  Verde }

Exemplo Google OR Tools



BACKTRACKING E SUDOKU

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9



ESCALONAMENTO BIO-INSPIRADO: ALGORITMOS GENÉTICOS

- **Indivíduo (Cromossomo):** Uma sequência completa de tarefas e horários.
- **Função de Aptidão (Fitness):** Minimizar tempo total de execução.
- **Operadores Evolutivos:**
 - **Seleção:** Escolher os melhores planos atuais.
 - **Crossover:** Misturar o início do Plano A com o fim do Plano B.
 - **Mutação:** Mudar aleatoriamente uma tarefa de lugar para evitar o "ótimo local".

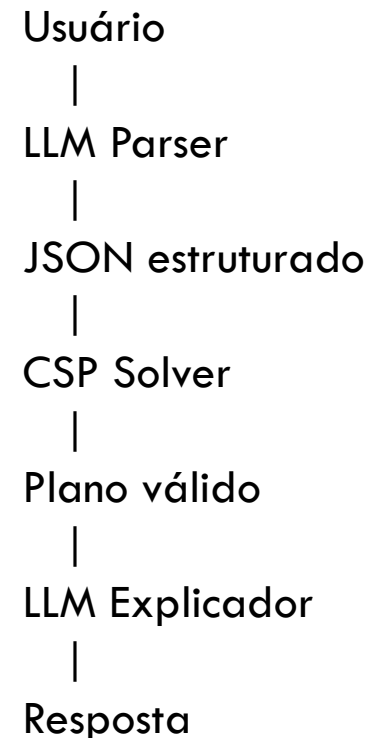
Vantagem: Encontra soluções excelentes em problemas de escala industrial (NP-Difíceis).

Exemplo PyGAD

IDEIA DE PROJETO 2

Mais difícil que modelar regras, mas LLMs também podem ser empregados para modelar produzir restrições (ou somente tarefas para planejamento se quiser ser mais simples).

Esquema geral com OR Tools



AGENTES MODERNOS DE IA

Os agentes modernos baseados em LLMs NÃO substituíram a IA clássica — eles estão reincorporando muitos conceitos clássicos de:

- planning;
- busca;
- memória;
- raciocínio simbólico;
- orquestração;
- decomposição de tarefas.

LLM

+

Ferramentas

+

Planning

+

Memória

+

Execução iterativa

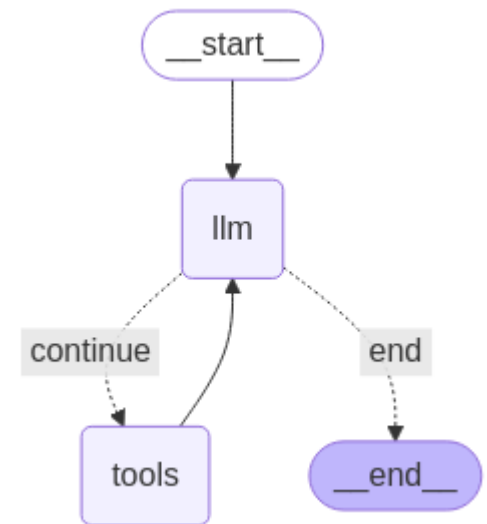
REACT

Ciclo Central: "Raciocínio-Agir-Observar"

Pensamento: O agente usa o raciocínio da Cadeia de Pensamento para analisar a tarefa, planejar os próximos passos e determinar se ela precisa de mais informações.

Ação: O agente executa uma ação específica.

Observação: O agente revisa os resultados dessa ação e realimenta seu raciocínio para decidir se ela está concluída ou precisa de outro ciclo.



LangGraph, LangChain, CrewAI

DISCUSSÕES

- Quais seriam os principais desafios (problemas) no uso de agentes de IA? Apresente ao menos 3 desafios explicando o porquê e dando um cenário de exemplo.
- Discuta o quê e como construir a IDEIA DE PROJETO I, apresentando uma ideia geral de solução/arquitetura.
- Discuta o quê e como construir a IDEIA DE PROJETO II, apresentando uma ideia geral de solução/arquitetura.
- Discuta: é algo útil inserirmos condições de falhas e/ou probabilidades/chances em um planejamento? Que estratégias/técnicas você empregaria para implementar isso sobre as soluções que aprendeu?